

Agentic RAG Engine

A Verified, Self-Correcting, Action-Capable
Knowledge System for Autonomous Knowledge Work

Project Proposal

Subhan Azhar

Software Engineering, Institute of Space Technology (IST)
August 2026

2	3	6+	7
systems fused: RAG + agents	new MCP servers	metrics tracked	build phases

Combining expert-level retrieval-augmented generation with expert-level
multi-agent orchestration, tool use, verification, and memory.

Abstract

This proposal outlines the design and implementation of **Agentic RAG Engine**, a multi-agent system that combines expert-level Retrieval-Augmented Generation (RAG) with expert-level agentic orchestration. Unlike common portfolio implementations that stop at a single embed-and-retrieve pipeline wrapped around a language model, Agentic RAG Engine is designed to plan, retrieve, reason, act through external tools via the Model Context Protocol (MCP), verify its own output, and maintain long-term memory across sessions. The system is built to be evaluated quantitatively rather than judged by demonstration alone, with a controlled experiment comparing naive RAG, hybrid RAG, and fully agentic RAG. This document details the system architecture, the specific techniques that distinguish it from basic RAG and basic agent implementations, the learning objectives it is designed around, the minimum viable feature set, and the build plan.

1 Motivation and Problem Statement

Most publicly available RAG and agentic AI projects share a common shape: documents are chunked, embedded, stored in a vector database, and retrieved by cosine similarity before being passed to a language model. This pattern is well understood, easy to implement, and no longer differentiates an engineer in a hiring context — it is the baseline, not the ceiling.

Two gaps motivate this project:

1. **RAG gap:** Naive retrieval degrades badly on real-world queries that are multi-hop, ambiguous, or phrased differently from the source documents. It has no mechanism to notice when retrieval has failed, and it produces answers with no verifiable link back to evidence.
2. **Agentic gap:** Most “agentic” demos are a single LLM call wrapped in a while-loop, with no explicit state, no error recovery, no tool discovery, and no safeguards on consequential actions. This does not reflect how production agent systems are actually engineered.

Agentic RAG Engine is designed specifically to close both gaps in one coherent system, so that the RAG subsystem and the agent subsystem reinforce each other rather than being two disconnected demos glued together.

2 What “Expert-Level” Means Here

2.1 Expert-level RAG: what is added beyond basic RAG

Capability	Basic RAG (not this)	Agentic RAG Engine (this project)
Retrieval method	Dense embeddings only	Hybrid: BM25 + dense, fused by reciprocal rank fusion
Result quality control	Top- k passed directly to the LLM	Retrieve top-30, cross-encoder reranker narrows to top-5
Query handling	Single query, verbatim	Query decomposition (multi-hop questions split into sub-questions) and query rewriting
Chunking	Fixed-size splitting	Semantic / parent-child chunking
Filtering	None	Metadata-aware filtering (date, source type, department, access)
Context handling	Retrieved text used as-is	Context compression before it reaches the LLM
Failure handling	Silent — bad retrieval yields a bad answer	Self-grading retrieval loop: rewrite and retry on failure
Trustworthiness	No link between answer and source	Citation verification against retrieved chunks
Evaluation	None, judged by eye	RAGAS-style metrics, measured and logged

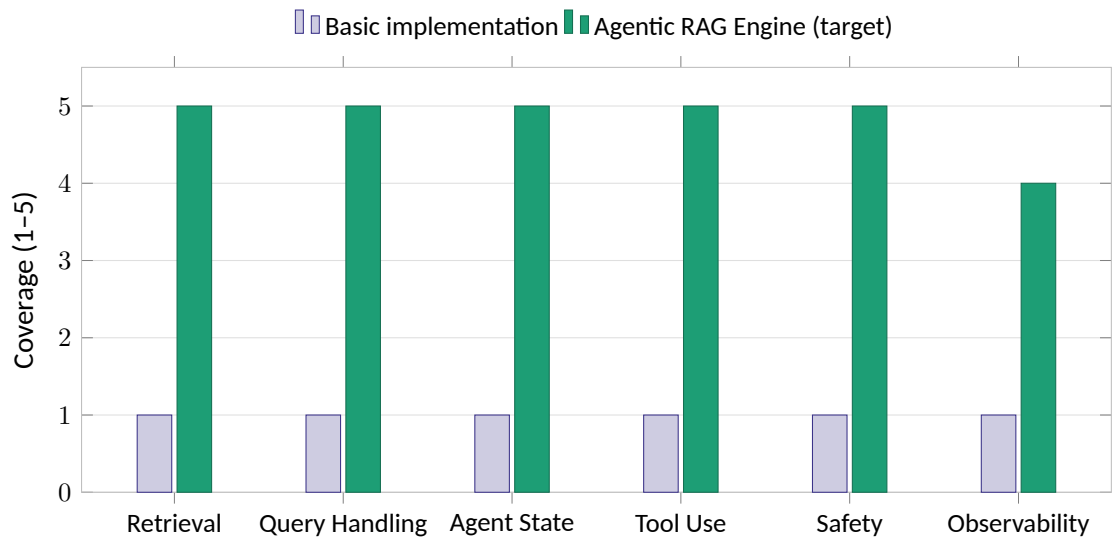
2.2 Expert-level agentic AI: what is added beyond basic agents

Capability	Basic agent (not this)	Agentic RAG Engine (this project)
Control flow	Single LLM call in a loop	Explicit multi-agent graph: orchestrator, research, planner, tool, critic
State	Implicit, in prompt history	Explicit shared state object (task, plan, evidence, tool results, errors, confidence)
Tool use	One or two hardcoded functions	MCP tool layer with dynamically discovered tools across custom servers
Error handling	Crashes or hallucinates on failure	Retry, fallback tool selection, or escalation to replanning
Self-checking	None; first answer is final	Critic/verifier agent gates the final answer; failure triggers re-plan
Action safety	All actions execute automatically	Risk-classified actions; destructive actions need human approval
Memory	None, or single-session only	Episodic memory (past sessions) and semantic memory (durable facts)
Observability	None	Full tracing of decisions, tool calls, and retries

The one-sentence test this project is designed to pass: *if you removed the RAG subsystem entirely, would the project still be an interesting agentic system? And if you removed the agents entirely, would the RAG subsystem still be interesting on its own?* Both answers should be yes — that is what makes this a genuine fusion project rather than a RAG demo with an agent bolted on, or an agent demo with a vector database bolted on.

3 Capability Comparison: Basic vs. Agentic RAG Engine

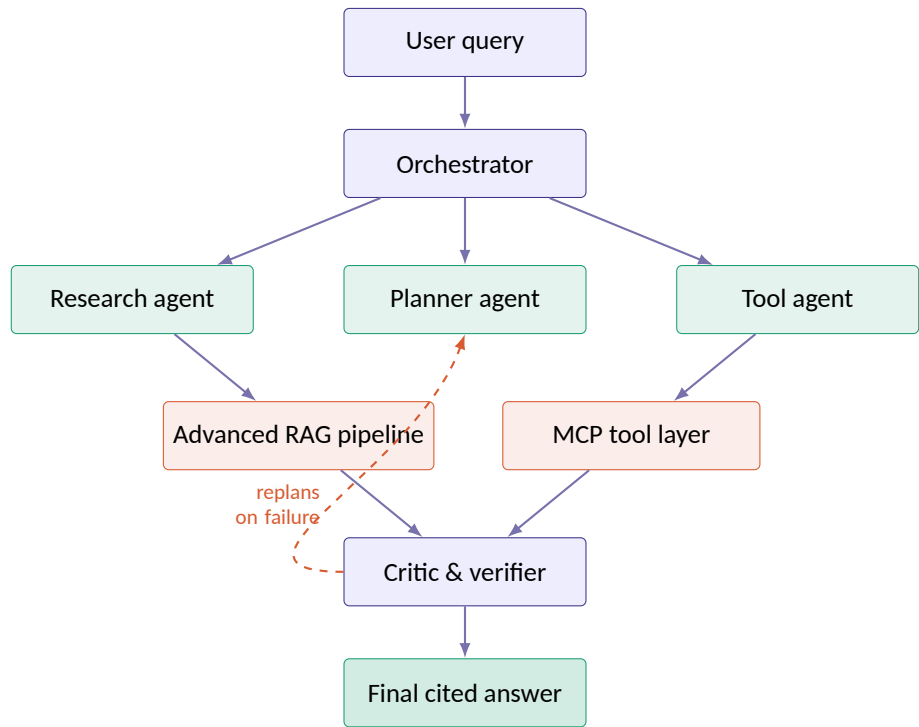
The chart below is a self-assessed coverage scale (1 = absent, 5 = fully implemented) used to plan scope — it is a design target, not a measured result.



4 System Architecture

4.1 High-level flow

A user query enters through an **orchestrator agent**, which routes it to a **research agent** (query decomposition and retrieval), a **planner agent** (multi-step execution planning), and a **tool agent** (execution through MCP). Retrieval flows through the **advanced RAG pipeline**; actions flow through the **MCP tool layer**. Both converge on a **critic/verifier agent**, which checks groundedness and citation quality before the system returns a **final, cited answer**. On failure, the critic triggers a replan loop rather than returning an unverified answer.



4.2 Shared agent state

```
class AgentState:
    task, plan, sub_questions
    evidence, tool_results
    intermediate_results, errors
    confidence, final_answer
```

Using an explicit, typed state object — rather than passing raw conversation history between agents — is the single detail that most distinguishes real agent engineering from a prompt-chaining script.

4.3 MCP tool layer

Rather than consuming a single pre-built MCP integration, this project builds **1–3 custom MCP servers** exposing real tools (e.g. `search_documents`, `run_sql`, `get_github_issue`, `create_issue`). The tool agent discovers available tools dynamically at runtime and selects among them, rather than calling one hardcoded function.

4.4 Human-in-the-loop safety layer

- **Read-only** actions (search, query, lookup) execute automatically.
- **Destructive** actions (create / modify / send) require a risk assessment and explicit human approval before the MCP tool executes.

4.5 Memory

- **Episodic memory** — record of past queries, actions taken, and outcomes.
- **Semantic memory** — durable facts the system has accumulated.

5 Evaluation Plan

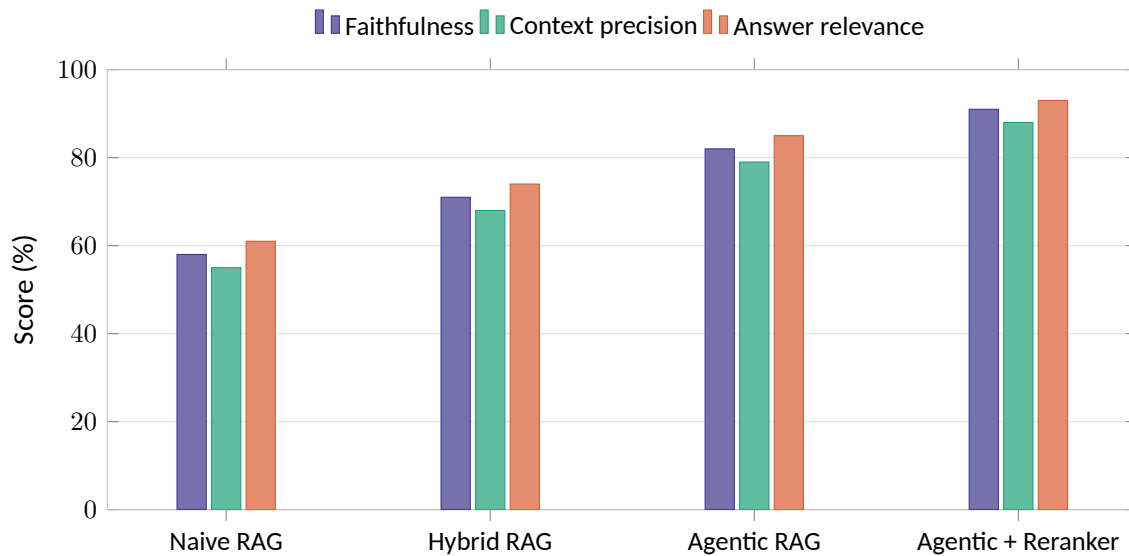
Unfinished or unmeasured agent projects are the norm; this project is explicitly scoped to produce numbers, not just a demo.

5.1 Metrics tracked

Metric	What it measures
Groundedness	Whether claims in the answer are supported by retrieved evidence
Retrieval recall	Whether the relevant chunks were retrieved at all
Answer accuracy	Whether the final answer is correct against a labelled benchmark
Tool success rate	Fraction of tool calls that complete without error or retry
Citation accuracy	Whether cited sources actually support the claims attached to them
Latency & cost	Average response time and token cost per query

5.2 Controlled experiment: target improvement trajectory

The chart below shows the *expected* trajectory the evaluation phase is designed to measure — illustrative planning targets, not yet-collected data.



5.3 Dashboard

A lightweight results dashboard summarizing the metrics above is included in the final deliverable.

6 Minimum Viable Feature Set

Must have (project is incomplete without these):

- Hybrid retrieval (BM25 + dense) with reranking
- Query decomposition and rewriting
- Multi-agent graph with explicit shared state (LangGraph or equivalent)
- At least one custom MCP server with dynamically discovered tools
- Critic/verifier agent with a working replan loop
- Human-in-the-loop approval on at least one destructive action
- An evaluation harness with the naive-vs-hybrid-vs-agentic comparison

Should have (strengthens the project, not blocking):

- Episodic and semantic memory layer
- A results dashboard (metrics table or simple UI)
- Metadata-aware filtering and context compression

Nice to have (stretch goals, time permitting):

- Multiple MCP servers spanning distinct domains (code, data, communication)
- A written failure-mode analysis documenting what broke and what was changed
- A small demo front end beyond a CLI or notebook

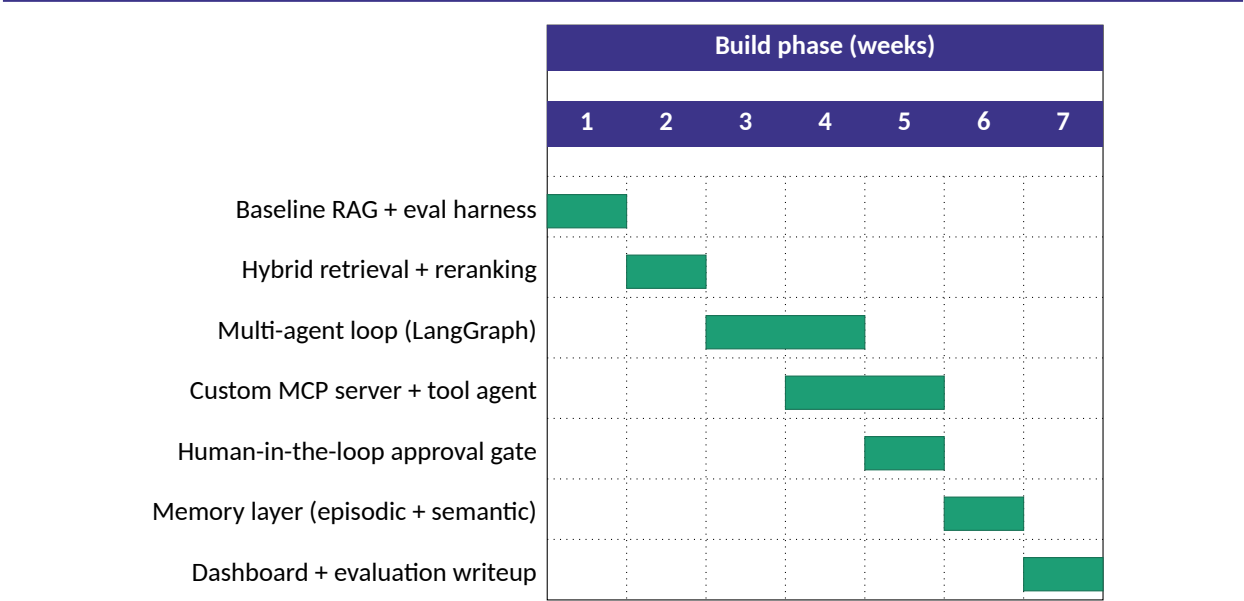
7 Technology Stack

Layer	Tooling
Orchestration	LangGraph / LangChain for the multi-agent state graph
Retrieval	Vector database (Qdrant / Weaviate), BM25 index, cross-encoder reranker
Tool integration	Model Context Protocol (MCP) — custom servers plus MCP client
Evaluation	RAGAS or an equivalent custom evaluation harness
Observability	Tracing via LangSmith / Langfuse or equivalent
Backend	FastAPI
Model access	OpenRouter or equivalent LLM API

8 Learning Objectives

1. **Advanced retrieval engineering** — hybrid search, reranking, query decomposition/rewriting, and why each step measurably improves retrieval quality.
2. **Multi-agent system design** — explicit state management, conditional graph routing, and agent-to-agent handoff.
3. **Tool-use and protocol design** — building and exposing tools through MCP, and reasoning about dynamic tool discovery and selection.
4. **Self-correcting systems** — designing feedback loops (critic → replan) rather than one-shot generation.
5. **AI safety and governance in practice** — risk classification of actions and human-in-the-loop approval gating.
6. **Memory system design** — the distinction between episodic and semantic memory and how each is retrieved and used.
7. **Rigorous evaluation** — building an evaluation harness and running controlled comparisons instead of judging output by inspection.
8. **Systems thinking under scope constraints** — shipping a finished, measured system rather than an unfinished ambitious one.

9 Build Plan and Timeline



1. Naive RAG baseline plus evaluation harness (establishes the number to improve against).
2. Hybrid retrieval and reranking — measure the delta against the baseline.
3. Multi-agent loop (research → plan → tool → critic) with explicit shared state.
4. One custom MCP server wired into the tool agent.
5. Human-in-the-loop approval gate on one destructive action.
6. Memory layer (episodic and semantic).
7. Results dashboard and the naive-vs-hybrid-vs-agentic writeup.

10 Expected Deliverables

- A working, deployed (or locally runnable) multi-agent RAG system meeting the minimum viable feature set in Section 5.
- A public code repository with a README containing the architecture diagram, evaluation table, and a documented failure-mode section.
- An evaluation report comparing naive RAG, hybrid RAG, and agentic RAG on the same benchmark queries.
- A short demo (recorded or live) showing the system handling a multi-hop query end to end, including a human-in-the-loop approval step.